NoMaD CARLA 障礙物測試：程式架構說明

Nov 7, 2026 · @Emma

總覽

這套程式在 CARLA 裡測試 NoMaD 能否只靠相機影像，開到 20 m 外的目標並繞過路上的障礙物。每次執行依序跑 4 個 trial：無障礙物（baseline）、機車（small）、小型車（medium）、廂型車（large），起點、目標和障礙物位置完全相同。

我們在原本的程式上加了 RGB-D 深度相機與 occupancy grid（提案第 3 節），用來檢查 NoMaD 的 8 條候選軌跡會不會撞到障礙物。**目前只做記錄，不影響車子的控制**；車子仍照原本 NoMaD 選出的路徑行駛。

整個測試只需要一個指令。所有參數都寫在 `general-navigation/carla_obstacle_test.sh` 裡，每個參數的意義見 repo 根目錄的 `README.md`。

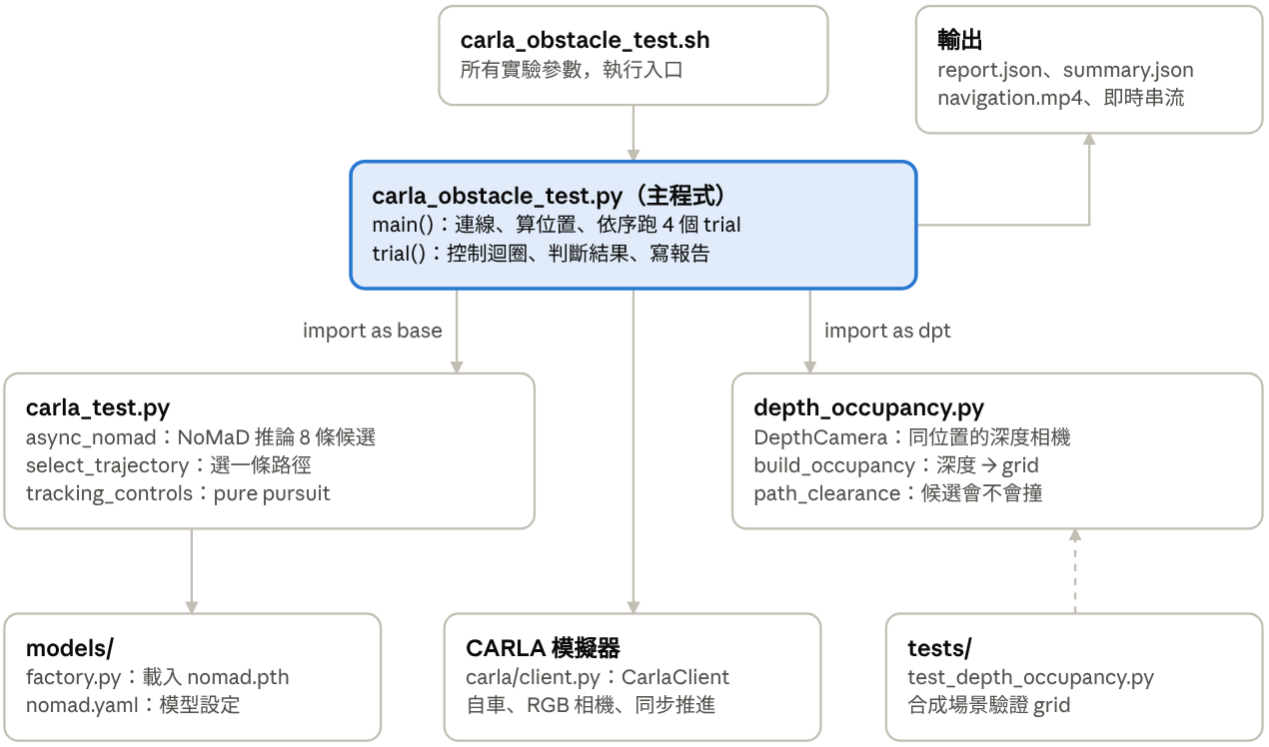
```
# 終端機 1：啟動 CARLA（等約 30 秒）
cd <CARLA 資料夾>
./CarlaUE4.sh -RenderOffScreen -quality-level=Low -nosound

# 終端機 2：執行測試
source ~/miniconda3/bin/activate ~/envs/nomad_carla
cd ~/ECE_780_NoMaD/general-navigation
bash carla_obstacle_test.sh

# 用http打開即時影像
http://localhost:8080/
```

執行時，主程式 `carla_obstacle_test.py` 負責整個實驗流程；它向 `carla_test.py` 借用 NoMaD 推論、路徑選擇和控制器，向 `depth_occupancy.py` 借用深度與 grid 功能。下方的圖說明這三個檔案之間的關係。

主程式負責流程，NoMaD 與深度功能各在一個檔案



程式架構 · 3 個主要檔案與周邊

實線箭頭是「呼叫」：主程式在中間，左邊負責 NoMaD，右邊負責深度；虛線是單元測試。

目錄結構

障礙物測試只用到 `general-navigation/` 裡的少數檔案；標 ★ 的是我們新增或修改過的。

```
ECE_780_NoMaD/
├── README.md                              ★ 執行方式與所有參數說明
├── carla                                  原作者電腦上的 CARLA 連結（壞的，不要用）
├── general-navigation/
│   ├── carla_obstacle_test.sh             ★ 執行腳本：所有參數寫在這裡
│   ├── requirements.txt                   Python 套件清單
│   ├── stream_server.py                   舊的獨立串流程式（障礙物測試不用）
│   ├── tests/
│   │   └── test_depth_occupancy.py        ★ 深度與 grid 的單元測試
│   └── general_navigation/
│       ├── carla_obstacle_test.py         ★ 主程式：障礙物實驗的完整流程
│       ├── depth_occupancy.py             ★ 新檔：深度相機 → occupancy grid → 碰撞檢
查
│       ├── carla_test.py                  NoMaD 推論、路徑選擇、控制器（被主程式引用）
│       ├── models/
│       │   ├── factory.py                 建立 NoMaD 模型、載入權重
│       │   └── config/nomad.yaml          模型設定（context_size、len_traj_pred
```

```
等）
|       └── carla/
|           ├── client.py            CarlaClient：生成自車、相機、推進模擬
|           └── helpers.py           CARLA 官方範例改寫的輔助類別（World、
CameraManager）
├── train/                           NoMaD / ViNT 訓練程式（本實驗不用）
├── deployment/                      實體機器人 ROS 部署（本實驗不用）
└── diffusion_policy/                diffusion policy 函式庫（被模型間接使用）
```

模型權重 `nomad.pth` 不在 repo 裡（`.gitignore` 排除了 `*.pth`），預設放在 `~/.cache/general_navigation/nomad.pth`。

carla_obstacle_test.py：主程式

這個檔案負責整個實驗：連線 CARLA、算出起點與障礙物位置、輪流跑 4 個 trial、評分並寫出結果。位置在 `general-navigation/general_navigation/`。

函式	做什麼
`main(args)`	連線 CARLA、載入地圖、建立自車與相機；沿道路算出目標點與障礙物點；把車移到目標拍 `goal.png`；載入模型；依序呼叫 4 次 `trial()`
`trial(...)`	跑一個完整的導航測試：生成障礙物、關閉迴圈控制、記錄影片與 `report.json`。核心流程見下方
`reset(client, t)`	把車瞬移到指定位置並確認停穩（位置誤差 > 1 m 或傾斜 > 15° 就報錯）
`spawn_obstacle(...)`	在障礙物 waypoint 上方 3 m 生成車輛、關閉物理、貼地放置
`navigation_outcome(...)`	每一步判斷結果：`arrived`（距目標 ≤ `goal-radius`）、`fail`（衝過目標線 ≥ `fail-distance`）、`stuck`（`stuck-seconds` 內移動 < `stuck-distance`）
`start_stream(port)` / `publish_frame(img)`	在 `localhost:<stream-port>` 提供 MJPEG 即時串流，內容和 `navigation.mp4` 相同
`pose_json(t)`	把 CARLA 位姿轉成 `[x, y, z, pitch, yaw, roll]` 寫進 JSON

幾個重要的幾何關係（全部沿道路量，不是直線距離）：

- 起點 = 地圖第 `spawn-index` 個 spawn point
- 目標 = 往前 `goal-distance` m（腳本設 20）
- 障礙物中心 = 往前 `goal-distance - obstacle-before-goal` m（腳本設 20 − 10 = 10）
- 障礙物清單寫在 `main()` 的 for 迴圈裡：`baseline`、`vehicle.harley-davidson.low_rider`、`vehicle.mini.cooper_s`、`vehicle.mercedes.sprinter`，要換障礙物就改那個清單
- 自車車型寫在 `main()` 的 `base.CarlaClient(...)`，預設 Tesla Model 3；可加 `filter='vehicle.micro.microlino'` 換小車

`trial()` 的主迴圈每圈代表 0.1 秒模擬時間，依序做：

1. `client.game_loop()` 推進模擬一步，記下這一幀車子的位姿
2. 開了 `--depth-camera` 時：取最新深度圖，用 `build_occupancy()` 建 grid
3. 把最新 RGB 影像放進 context（最近 4 張、間隔固定 0.1 秒），湊滿就送去推論子行程
4. 收到 NoMaD 的 8 條候選 → `select_trajectory()` 選一條；開了深度時同時檢查 8 條會不會撞
5. `tracking_controls()`（pure pursuit）算出油門與轉向；太久沒有新預測就停車
6. `navigation_outcome()` 判斷是否結束；畫出影片這一格、更新串流
7. 套用控制量，進入下一圈

結束後（即使出錯）先存好影片，再停車、關閉推論、清除感測器與障礙物，最後寫出 `report.json`。

depth_occupancy.py：深度 → occupancy grid

這個新檔案用 CARLA 的深度相機取代提案中的 Depth Anything V2，把深度圖變成機器人周圍的 2D 佔據地圖，再拿來檢查候選軌跡。所有數學都是純 numpy，可以用 `tests/test_depth_occupancy.py` 單獨測試，不需要 CARLA。

處理順序（對應提案第 3 節的公式）：

1. `decode_carla_depth()`：CARLA 的 RGB 編碼 → 公尺（沿相機正前方的距離，即 Z_c）
2. `intrinsics_from_fov()`：由 FOV 算內參，256×128、FOV 90° 時 $f_x = f_y = 128$
3. `back_project()`：每個像素 (u, v) → 相機座標 3D 點 P_c
4. `camera_extrinsics()`：由相機安裝位置算外參 R、t，得到 $P_r = R\,P_c + t$
5. `build_occupancy()`：高度過濾後算出格子索引，產生 `occupied` 與 `observed` 兩張地圖

Source

$$i = \left\lfloor \frac{X_r - X_{min}}{r} \right\rfloor, \quad j = \left\lfloor \frac{Y_r - Y_{min}}{r} \right\rfloor, \quad 0 \le i < N_x,\ 0 \le j < N_y$$

超出範圍的點直接丟掉；(0, 0) 在自車的右後角，自車中心在 (20, 80)。

三種座標系（最容易搞錯的地方）：

座標系	x	y	z
相機光學座標（提案）	右	下	前
CARLA 世界 / UE4	前	右	上
機器人 / NoMaD（grid 用這個）	前	左	上（從路面量）

Grid 預設設定（`GridConfig`，可用參數覆蓋）：解析度 0.1 m；x 從車後 2 m 到車前 20 m；y 左右各 8 m；只保留離地 0.2–2.5 m 的點；超過 25 m 的深度丟掉。

`OccupancyGrid` 的方法：

方法	用途
`inflated(radius)`	障礙物往外膨脹；半徑 = 自車半寬 + `safety-margin`，只要檢查中心線就等於檢查整個車寬
`path_clearance(path)`	一條軌跡會不會撞、離最近障礙物多遠
`front_clearance()`	車頭前方、車寬內，到最近障礙物的距離
`distance_field()`	每格到最近障礙物的距離（OpenCV distance transform）

其他函式：

- `DepthCamera`：生成和 RGB 相機同位置、同解析度、同 FOV 的深度相機，依 frame 編號保存最近 32 張深度圖。另外生成，是因為 `CameraManager` 一次只能有一個感測器
- `extend_path(path, horizon)`：只在碰撞檢查時，把候選沿最後一段方向延伸到 `check-horizon` m。NoMaD 的 8 個 waypoint 只有約 2 m，不延伸會太晚才發現障礙物
- `colorize_depth()`、`render_bev()`：只用來畫影片的中間格（彩色深度，紅 = 近）和右格（BEV：白 = 障礙物、紅 = 膨脹區、黃線 = 安全候選、紅線 = 會撞、藍線 = 選中、綠框 = 自車）

carla_test.py：NoMaD 推論與控制（被主程式引用）

主程式以 `base` 的名稱匯入這個檔案，借用裡面的模型推論、路徑選擇和控制器。它本身也是另一個可以單獨執行的程式（用網頁手動開車、拍目標圖），但障礙物測試不會執行它的 `main()`。

會影響實驗結果的常數（檔案開頭）：

常數	值	意義
`WAYPOINT_SPACING_METERS`	0.25	NoMaD 輸出換算成公尺的比例；8 個 waypoint 約往前 2 m
`NUM_SAMPLES`	8	每次推論產生的候選軌跡數
`MAX_SPEED`	0.6	最高油門；轉越多速度越慢
`MIN_LOOKAHEAD`	1.5 m	pure pursuit 至少看多遠的點
`STEER_RATE`	0.5 / s	每秒最多改變多少轉向
`CONTROL_DT`	0.1 s	每個控制步驟的模擬時間

被主程式使用的函式：

函式	做什麼
`async_nomad()`	在另一個子行程跑 NoMaD：收影像、做 diffusion 取樣，回傳 8 條候選。推論再慢也不會卡住控制迴圈
`select_trajectory()`	從 8 條候選選一條。`current`：選和其他候選最相近的（consensus），加上和上一條的連續性；`official`：直接用第一條。**這裡就是之後要接上安全過濾器的地方**
`tracking_controls()`	pure pursuit 控制器：由路徑算出轉向與速度
`FixedImageContext`	保存模擬時間間隔固定 0.1 秒的歷史影像；中間漏幀就清空重來
`local_to_world()` / `world_to_local()`	NoMaD 局部座標（x 前、y 左）與 CARLA 世界座標互轉
`vehicle_pose()` / `vehicle_geometry()`	車子的 (x, y, yaw)；車軸距與最大轉向角
`camera_state()`	取得最新一幀 RGB 影像

函式	做什麼
`apply_vehicle_controls()` / `stopped_controls()`	把控制量送給車子；停車指令
`draw_nomad_paths()`	在 RGB 影像上畫黃線（候選）和藍線（選中）。它不知道碰撞檢查結果，所以這裡永遠沒有紅線

`CarlaClient` 實際定義在 `carla/client.py`，經由這個檔案匯入。

其他檔案

這些檔案被間接用到，或者只在特定情況需要修改。

檔案	做什麼	什麼時候會動到
`general-navigation/carla_obstacle_test.sh`	把所有參數傳給主程式	每次改實驗設定；`--checkpoint` 與 `--output` 必須改成自己的路徑
`README.md`	執行方式與每個參數的意義	查參數
`models/factory.py`	`get_model()` 建立 NoMaD；`get_weights()` 載入 `nomad.pth`（沒給 `--checkpoint` 時會自動下載到 `~/.cache`）	權重載入失敗時
`models/config/nomad.yaml`	模型設定：`context_size: 3`（看 4 張影像）、`len_traj_pred: 8`（8 個 waypoint）	不要改，模型是用這組設定訓練的
`carla/client.py`	`CarlaClient`：生成自車（預設 `vehicle.tesla.model3`）、256×128 RGB 相機、同步模式推進	換自車、改影像解析度
`carla/helpers.py`	CARLA 官方 `manual_control.py` 改寫的 `World`、`CameraManager`、`HUD`	很少；相機安裝位置在 `CameraManager` 裡
`tests/test_depth_occupancy.py`	用合成場景測試深度解碼、座標轉換、grid 與碰撞檢查	改了 `depth_occupancy.py` 之後執行 `python tests/test_depth_occupancy.py`
`stream_server.py`	舊的獨立串流程式，會掛到場上第一台車	不用；障礙物測試有自己的 `--stream-port`

檔案	做什麼	什麼時候會動到
`train/`、`deployment/`	NoMaD 訓練程式、實體機器人 ROS 部署	本實驗不用
`diffusion_policy/`	diffusion policy 函式庫	不用動，模型內部會用到

輸出檔案

每次執行的結果都寫在 `--output` 指定的資料夾；資料夾裡已經有舊結果時，會自動建一個 `run_<時間戳>` 子資料夾，不會覆蓋。

```
<output>/run_20261107_094641_128056/
├── goal.png              NoMaD 看到的目標影像
├── manifest.json         參數、位姿、程式碼 hash（重現實驗用）
├── summary.json          4 個 trial 的結果摘要
├── error.txt             出錯時才有
└── baseline/ small/ medium/ large/
    ├── navigation.mp4    三格並排：RGB | 彩色深度 | BEV
    ├── initial.png / last.png
    ├── report.json       逐步狀態、每次預測、碰撞紀錄
    └── depth/*.npz       開了 --save-depth 才有
```

`report.json` 的重要欄位：

欄位	意義
`outcome` / `failure_reason` / `stuck_reason`	結果：`arrived`、`fail`（如 `goal_overshoot`）、`stuck`（如 `obstacle_contact`）
`arrived` / `reached_without_collision`	有沒有抵達；是否在沒碰撞的情況下抵達
`closest_distance_m` / `final_distance_m` / `path_length_m`	最接近目標的距離、最後距離、實際行駛距離
`collisions`	每次碰撞撞到什麼、是不是障礙物
`states[].goal_overshoot_m`	沿道路方向越過目標線多少公尺（負 = 還沒到）
`states[].front_clearance_m`	車頭到車道內最近障礙物的距離
`predictions[].inference_seconds`	每次推論花多久（GPU 約 0.1 秒；好幾秒代表在用 CPU）
`predictions[].candidate_collides`	8 條候選各自會不會撞

欄位	意義
`predictions[].candidate_clearance_m`	8 條候選各自離障礙物多遠
`predictions[].feasible_count`	安全候選數；等於 0 的比例 = 提案的 fallback activation rate
`depth_camera`	內參、外參 R、t、grid 參數、自車尺寸 `ego_box`（車尾 x、車頭 x、半寬）

常見問題

下面是實際跑過遇到的狀況。大部分問題出在 CARLA 或環境，不是程式本身。

症狀	原因	解法
`python: command not found`	沒啟動 conda 環境	`source ~/miniconda3/bin/activate ~/envs/nomad_carla`，每開一個終端機都要
`Checkpoint not found`	`--checkpoint` 路徑錯，或不是絕對路徑	改成 `nomad.pth` 的絕對路徑；沒有就 `gdown 1YJhkkMJAYOiKNyCaelbS_alpUpAJsOUb -O <路徑>`
`bind: Address already in use`，CARLA 立刻 segfault	port 2000 已有 CARLA（通常是自己開過的）	`ps -eo user,pid,etime,cmd | grep -i [C]arlaUE4`；是自己的就直接用或 `pkill -u $USER -f CarlaUE4`；是別人的就用 `-carla-rpc-port=2010` 配 `--carla-port 2010`
卡在 `INIT: world_settings`	CARLA 狀態異常（上次被 Ctrl+C 中斷），或連到別人的 CARLA	重啟 CARLA，等 30 秒再跑
`CUDA unknown error`、載入權重時在 `torch.load` 失敗	CARLA 剛啟動時 PyTorch 搭不到 GPU	等 CARLA 穩定後重跑；用 `python -c "import torch; torch.zeros(1).cuda()"` 確認 GPU
`channel 3: open failed: connect failed`	瀏覽器在連 8080，但串流還沒啟動	無害，程式跑起來就會消失
串流看不到	8080 被同台 server 的其他人佔用	`--stream-port 8091`，ssh 改成 `-L 8080:localhost:8091`

症狀	原因	解法
不能寫入 `--output`	腳本裡留著別人的家目錄路徑	改成 `$HOME/carla_test_results/...`
`cd .../carla` 失敗	repo 裡的 `carla` 是原作者電腦的連結	用自己 CARLA 的實際路徑；`find ~ -name CarlaUE4.sh`
BEV 裡沒有紅線	NoMaD 候選只有約 2 m，還碰不到障礙物	加 `--check-horizon 6`；RGB 畫面本來就只有黃、藍線
`goal_overshoot`	繞過障礙物後停在隔壁車道，經過目標時橫向距離 > 2 m	正常的失敗類別；可調大 `--goal-radius` 或 `--fail-distance`
有繞但繞不夠、最後 `stuck`	Tesla 寬約 2.2 m，NoMaD 學的是小機器人的繞行幅度	試 `--path-selection official` 或自車換 `vehicle.micro.microlino`

CARLA Python client 的版本要和模擬器一致（用 `pip show carla` 對照）；不一致會印出 `Version mismatch` 警告，可能導致難以判斷的錯誤。